

CustomGate360

Documentación Técnica del Backend — v2.0

Sistema de Gestión Aduanera con Motor de Perfilamiento de Riesgo

■ Actualizado: tabla lista_negra_global · 4to JOIN · Relaciones y cardinalidades completas

Stack tecnológico	Spring Boot 3 + PostgreSQL + Angular 20
Arquitectura	Controller → Service → Repository (MVC)
Motor de riesgo	SQL con 4 LEFT JOINs simultáneos
Base de datos	PostgreSQL · 9 tablas · ddl-auto=update
Score máximo	150 pts (actualizado con lista negra)
Puerto backend	localhost:8080

1. Arquitectura General del Proyecto

El proyecto sigue el patrón MVC dividido en dos aplicaciones independientes que se comunican por HTTP REST:

Capa	Tecnología	Puerto	Responsabilidad
Frontend	Angular 20 (TypeScript)	4200	Interfaz de usuario, formularios, visualización
Backend	Spring Boot 3 (Java 21)	8080	API REST, lógica de negocio, acceso a BD
Base de datos	PostgreSQL	5431	Persistencia de datos, cálculos SQL complejos

Estructura de paquetes del Backend

Subcarpeta	Descripción
BackendApplication.java	Punto de entrada. main() que arranca Spring Boot con @SpringBootApplication
config/	CorsConfig.java y DataInitializer.java — CORS y datos semilla al arrancar
controllers/	Reciben peticiones HTTP y delegan al service o repositorio. 6 controllers
dto/	Clases de transferencia de datos. NO son tablas BD. Solo empaquetan respuestas HTTP
models/	Entidades JPA: cada @Entity = una tabla en PostgreSQL. 9 entidades (incluye lista_negra_global)
repositories/	Interfaces Spring Data JPA. SQL CRUD automático. 9 repositorios
services/	Lógica de negocio: motor de riesgo y estadísticas. 2 services

2. Subcarpeta: config/

CorsConfig.java

Permite peticiones de Angular (localhost:4200) y Vercel (*.vercel.app) al backend:

Parámetro	Valor
Orígenes permitidos	http://localhost:4200 · https://*.vercel.app · https://proyecto-ing-web.onrender.com
Métodos HTTP	GET, POST, PUT, DELETE, OPTIONS
Headers	Todos (*)
Credenciales	allowCredentials=true

DataInitializer.java — CommandLineRunner

Tabla	Estrategia	Datos sembrados
catalogo_riesgo_pais	DELETE ALL + recarga	13 puertos: Ecuador (4), España (4), USA (5)
importador_historial	Solo si vacía	8 empresas, 2 infractoras (>2 infracciones)
restricciones_arancelarias	Solo si vacía	12 códigos arancelarios, 6 requieren permiso especial
lista_negra_global	Solo si vacía	5 entidades: 2 coinciden con importadores del sistema (OFAC/ONU)
pais / provincia / ciudad	Solo si pais vacía	Ecuador, USA, España con provincias y ciudades

3. Subcarpeta: models/ (9 Entidades JPA → 9 Tablas PostgreSQL)

Cada clase @Entity es convertida por Hibernate en una tabla PostgreSQL. ddl-auto=update crea/actualiza tablas automáticamente al iniciar.

Clase Java	Tabla BD	Campos principales	Notas
OperacionAduanera	operaciones_aduaneras	id, numeroTracking (UQ), tipoOperacion, estado, puertoOrigen, puertoDestino, fechaRegistro, idImportador, codigoArancelario, canalAforo, puntajeRiesgo	Tabla central. canalAforo y puntajeRiesgo son CALC por el motor de riesgo
Usuario	usuario	id, username (UQ), password, rol, cedula (UQ,10 dígitos), nombreCompleto	@NotBlank y @Pattern. rol=ADMIN por defecto
CatalogoRiesgoPais	catalogo_riesgo_pais	id, nombrePuertoOPais, nivelRiesgo (ALTO/MEDIO/BAJO), puntos (0/15/30), motivo	Vector 1 del JOIN. Se recarga completo en cada inicio
ImportadorHistorial	importador_historial	id, nombreEmpresa, rucEmpresa (UQ), infraccionesPrevias, paisOrigen	Vector 2 del JOIN. >2 infracciones = +40 pts. rucEmpresa se cruza con lista_negra_global
RestriccionArancelaria	restricciones_arancelarias	id, codigoArancelario (UQ), descripcion, requierePermiso (boolean), categoria	Vector 3 del JOIN. requierePermiso=true = +30 pts
ListaNegraGlobal	lista_negra_global	id, rucSancionado (UQ), nombreEntidad, organismoSancionador, motivoSancion, puntosExtra	NUEVO — Vector 4. Se cruza con importador_historial.rucEmpresa. Presencia = +50 pts
Pais	pais	id, nombre	Tabla de ubicación. Usa @Data de Lombok
Provincia	provincia	id, nombre, pais_id (FK)	@ManyToOne con Pais. 1:N
Ciudad	ciudad	id, nombre, provincia_id (FK)	@ManyToOne con Provincia. 1:N

4. Subcarpeta: repositories/

Todos extienden **JpaRepository<Entidad, Long>**. Spring Data JPA genera automáticamente `findAll()`, `findById()`, `save()`, `deleteById()`, `count()`, `existsById()`.

Repositorio	Métodos adicionales
OperacionAduaneraRepository	<code>findByNumeroTracking(String)</code> → <code>SELECT WHERE numero_tracking = ?</code>
CatalogoRiesgoPaisRepository	<code>findByNombrePuertoOPaisIgnoreCase(String)</code>
ImportadorHistorialRepository	Solo métodos base de JpaRepository
RestriccionArancelariaRepository	Solo métodos base
ListaNegraGlobalRepository	Solo métodos base — NUEVO
UsuarioRepository	<code>findByUsername(String)</code> → usado en el login
PaisRepository	Solo métodos base
ProvinciaRepository	<code>findByPaisId(Long)</code> → filtra provincias por país
CiudadRepository	<code>findByProvinciaId(Long)</code> → filtra ciudades por provincia

5. Subcarpeta: dto/ (Data Transfer Objects)

Los DTOs son clases que solo transportan datos. No tienen @Entity, no son tablas, no tienen lógica de negocio.

DetalleRiesgoDTO.java — Actualizado con Vector 4

Bloque	Campos	Tabla de origen
Vector 1: Origen geográfico	nivelRiesgoPais, puntosOrigen, motivoPais, origenEnCatalogo	catalogo_riesgo_pais
Vector 2: Historial importador	nombreImportador, infraccionesImportador, puntosImportador, importadorInfractor	importador_historial
Vector 3: Mercancía arancelaria	descripcionMercancia, categoriaMercancia, requierePermiso, puntosMercancia	restricciones_arancelarias
Vector 4: Lista Negra Global	importadorEnListaNegra, organismoSancionador, motivoSancion, puntosListaNegra	lista_negra_global (JOIN con importador)
Resultado final	puntajeTotal, canalAforo, descripcionCanal	Calculado en Java con los 4 vectores

OperacionConRiesgoResponse.java

Campo	Tipo	Descripción
operacion	OperacionAduanera	Entidad guardada con canalAforo y puntajeRiesgo ya asignados
detalleRiesgo	DetalleRiesgoDTO	Desglose de los 4 vectores: qué encontró y cuántos puntos sumó cada uno

EstadisticasDTO.java

Grupo	Campos
Producto más solicitado	productoCodigo, productoDescripcion, productoCategoria, productoTotalOperaciones
Puerto más riesgoso	puertoNombre, puertoNivelRiesgo, puertoTotalOperaciones, puertoOperacionesRiesgo
Empresa más infractora	empresaNombre, empresaInfracciones, empresaPaisOrigen
Totales generales	totalOperaciones, operacionesVerdes, operacionesAmarillos, operacionesRojas

6. Subcarpeta: services/

PerfilRiesgoService.java — Motor con 4 LEFT JOINs (actualizado)

Usa **NamedParameterJdbcTemplate** para ejecutar SQL puro. Ahora con **4 LEFT JOINs** simultáneos. El score máximo subió a **150 pts**:

Canal	Rango	Acción
VERDE	0 – 30 pts	Desaduanización Automática: estado pasa a DESADUANIZACION sin intervención
AMARILLO	31 – 70 pts	Aforo Documental: Agente sube Factura, Certificado de Origen, Póliza
ROJO	71 – 150 pts	Aforo Físico Intrusivo: Inspector abre contenedor y llena reporte

Vector	Tabla cruzada	Condición	Puntos sumados
1 — Origen	catalogo_riesgo_pais	LOWER(TRIM) del puerto_origen = nombre_puerto_o_pais	0 / 15 / 30 según nivel
2 — Importador	importador_historial	ih.id = :idImportador	40 si infracciones_previas > 2
3 — Mercancía	restricciones_arancelarias	LOWER(TRIM) del codigo_arancelario	30 si requiere_permiso = true
4 — Lista Negra	lista_negra_global	LOWER(TRIM) lng.ruc_sancionado = LOWER(TRIM) ih.ruc_empresa	50 si importador está sancionado

EstadisticasService.java

Consulta	SQL	Resultado
Totales por canal	COUNT + SUM(CASE WHEN canal_aforo=...) FROM operaciones_aduaneras	Cuántas son VERDE, AMARILLO, ROJO
Producto más solicitado	GROUP BY codigo_arancelario + LEFT JOIN restricciones ORDER BY COUNT DESC LIMIT 1	Código arancelario más frecuente
Puerto más riesgoso	GROUP BY puerto_origen + LEFT JOIN catalogo, contando ROJO+AMARILLO	Puerto con más ops de riesgo
Empresa más infractora	ORDER BY infracciones_previas DESC LIMIT 1	Empresa del historial con más infracciones

7. Subcarpeta: controllers/

Controller	Base URL	Endpoints principales
OperacionAduaneraController	/api/operaciones	GET / · POST / (motor 4 JOINS) · GET /tracking/{nro} · PUT /{id}/estado · GET /{id}/analisis · GET /alerta-roja · DELETE /{id}
CatalogoController	/api/catalogos	GET+POST /países-riesgo · GET+POST /importadores · GET+POST /arancelarios · GET+POST /lista-negra (NUEVO)
UsuarioController	/api/admin	POST /login · POST /registro · GET /listar · DELETE /{id}
UbicacionController	/api/ubicaciones	GET /países · GET /provincias/{paisId} · GET /ciudades/{provincialId}
EstadisticasController	/api/estadisticas	GET / → devuelve EstadisticasDTO con 4 métricas
ApiController	/api	GET / → health check básico

8. Relaciones entre Tablas SQL y Cardinalidades

8.1 Foreign Keys explícitas — @ManyToOne en JPA

Tabla hija	Campo FK	Tabla padre	Cardinalidad	Significado
provincia	pais_id	pais	1 : N	Un país tiene MUCHAS provincias. Una provincia pertenece a UN solo país
ciudad	provincia_id	provincia	1 : N	Una provincia tiene MUCHAS ciudades. Una ciudad pertenece a UNA sola provincia

8.2 Relaciones lógicas — JOINS del motor de riesgo (sin FK en BD)

Estas relaciones no tienen FK formal. Se cruzan en runtime mediante LEFT JOIN. El motivo es que si un puerto/importador/código no existe en el catálogo, el LEFT JOIN devuelve NULL en vez de un error de integridad referencial — esto es una decisión de diseño intencional.

Tabla origen	Campo	Tabla cruzada	Cardinalidad	Condición JOIN	Puntos
operaciones_aduaneras	puerto_origen	catalogo_riesgo_pais	N : 1	LOWER(TRIM) match en nombre_puerto_o_pais	0/15/30
operaciones_aduaneras	id_importador	importador_historial	N : 1	ih.id = :idImportador (join por ID)	0 o 40
operaciones_aduaneras	codigo_arancelario	restricciones_arancelarias	N : 1	LOWER(TRIM) match en codigo_arancelario	0 o 30
importador_historial	ruc_empresa	lista_negra_global	1 : 0..1	LOWER(TRIM) Ing.ruc_sancionado = ih.ruc_empresa (JOIN entre catálogos)	0 o 50

8.3 Tabla de todas las cardinalidades

Relación	Tipo	Descripción
pais → provincia	1 : N	Un país tiene N provincias. Implementada con @ManyToOne + pais_id FK real en BD
provincia → ciudad	1 : N	Una provincia tiene N ciudades. Implementada con @ManyToOne + provincia_id FK real en BD
operaciones → catalogo_riesgo_pais	N : 1	Muchas operaciones pueden tener el mismo puerto de origen
operaciones → importador_historial	N : 1	Muchas operaciones pueden pertenecer al mismo importador
operaciones → restricciones_arancelarias	N : 1	Muchas operaciones pueden tener el mismo código arancelario
importador_historial → lista_negra_global	1 : 0..1	Un importador puede estar (1) o no estar (0) en la lista negra. Relación opcional
usuario	Independiente	Sin relaciones FK. Tabla aislada para autenticación

9. La Consulta SQL Central: 4 LEFT JOINs Simultáneos

Esta es la consulta más importante del sistema. Se ejecuta en **PerfilRiesgoService** cada vez que se crea o analiza una operación. El Vector 4 es el más destacado porque **cruza dos tablas de catálogo entre sí** (importador_historial ↔ lista_negra_global):

```
SELECT
  -- Vector 1: catalogo_riesgo_pais
  CASE WHEN crp.id IS NOT NULL THEN true ELSE false END AS origen_en_catalogo,
  COALESCE(crp.nivel_riesgo, 'N/A') AS nivel_riesgo_pais,
  COALESCE(crp.puntos, 0) AS puntos_origen,

  -- Vector 2: importador_historial
  COALESCE(ih.nombre_empresa, 'Sin importador') AS nombre_importador,
  COALESCE(ih.infracciones_previas, 0) AS infracciones_importador,
  CASE WHEN COALESCE(ih.infracciones_previas,0) > 2
    THEN 40 ELSE 0 END AS puntos_importador,

  -- Vector 3: restricciones_arancelarias
  COALESCE(ra.descripcion, 'Mercancia general') AS descripcion_mercancia,
  COALESCE(ra.requiere_permiso, false) AS requiere_permiso,
  CASE WHEN COALESCE(ra.requiere_permiso,false) = true
    THEN 30 ELSE 0 END AS puntos_mercancia,

  -- Vector 4: importador_historial vs lista_negra_global
  -- (JOIN entre DOS catálogos – el más destacado)
  CASE WHEN lng.id IS NOT NULL THEN true ELSE false END AS importador_en_lista_negra,
  COALESCE(lng.organismo_sancionador, 'N/A') AS organismo_sancionador,
  CASE WHEN lng.id IS NOT NULL
    THEN COALESCE(lng.puntos_extra, 50) ELSE 0 END AS puntos_lista_negra,

  -- PUNTAJE TOTAL (4 vectores en BD – máximo 150 pts)
  (COALESCE(crp.puntos, 0)
  + CASE WHEN COALESCE(ih.infracciones_previas,0)>2 THEN 40 ELSE 0 END
  + CASE WHEN COALESCE(ra.requiere_permiso,false)=true THEN 30 ELSE 0 END
  + CASE WHEN lng.id IS NOT NULL THEN COALESCE(lng.puntos_extra,50) ELSE 0 END
  ) AS puntaje_total

FROM (VALUES (1)) AS dummy(x)

LEFT JOIN catalogo_riesgo_pais crp
  ON LOWER(TRIM(crp.nombre_puerto_o_pais)) = LOWER(TRIM(:puertoOrigen))

LEFT JOIN importador_historial ih
  ON ih.id = :idImportador

LEFT JOIN restricciones_arancelarias ra
  ON LOWER(TRIM(ra.codigo_arancelario)) = LOWER(TRIM(:codigoArancelario))

LEFT JOIN lista_negra_global lng
  ON LOWER(TRIM(lng.ruc_sancionado)) = LOWER(TRIM(COALESCE(ih.ruc_empresa,'')))
```

Vector	Condición	Puntos	Máximo acumulado
1 — Origen ALTO	Puerto en catálogo con nivel ALTO	30	30
1 — Origen MEDIO	Puerto en catálogo con nivel MEDIO	15	

1 — Origen BAJO/N/A	Puerto no catalogado o nivel BAJO	0	
2 — Importador	infracciones_previas > 2	40	70
3 — Mercancía	requiere_permiso = true	30	100
4 — Lista negra	Importador aparece en lista_negra_global	50	150
MÁXIMO POSIBLE			150

10. Flujo Completo: Crear una Operación Aduanera

Pa so	Actor	Acción
1	Frontend Angular	POST /api/operaciones con JSON de la operación
2	OperacionAduaneraController	Recibe @RequestBody. Fuerza estado = 'DOCUMENTACION'
3	PerfilRiesgoService	Ejecuta SQL con 4 LEFT JOINS en PostgreSQL. Obtiene puntaje_total y detalles
4	PerfilRiesgoService	Calcula canal: 0-30=VERDE, 31-70=AMARILLO, 71-150=ROJO. Si VERDE → estado='DESADUANIZACION'
5	PerfilRiesgoService	Asigna canalAforo y puntajeRiesgo a la entidad en memoria
6	OperacionAduaneraController	Persiste operación via operacionRepository.save()
7	OperacionAduaneraController	Devuelve OperacionConRiesgoResponse (operación + DetalleRiesgoDTO con 4 vectores)
8	Frontend Angular	Muestra panel con los 4 vectores: origen, importador, mercancía y lista negra

Estados posibles de una operación

Estado	Descripción	Quién asigna
DOCUMENTACION	Estado inicial	Automático al crear
DESADUANIZACION	Canal VERDE: sin revisión humana	Automático si score $\leq$ 30
LOGISTICA	Coordinación logística	Manual — PUT /{id}/estado
EN_TRANSITO	Carga en tránsito	Manual
LLEGADA_PUERTO	Carga llegó al puerto de destino	Manual
CERRADO	Operación finalizada	Manual

11. Cómo Agregar una Tabla Nueva (Guía para la defensa)

Paso	Archivo	Qué hacer
1 — Modelo	models/NuevaEntidad.java	@Entity + @Table + campos + getters/setters. Hibernate crea la tabla automáticamente (ddl-auto=update)
2 — Repository	repositories/NuevaEntidadRepository.java	public interface NuevaEntidadRepository extends JpaRepository {}
3 — Controller	controllers/CatalogoController.java	Agregar @Autowired del repo + @GetMapping + @PostMapping
4 — JOIN (opcional)	services/PerfilRiesgoService.java	Agregar LEFT JOIN adicional al SQL_ANALISIS_RIESGO y CASE WHEN en el puntaje_total
5 — DTO (opcional)	dto/DetalleRiesgoDTO.java	Agregar campos del nuevo vector y sus getters/setters

12. Resumen Completo de Endpoints REST

Método	Endpoint	Descripción
GET	/api/operaciones	Lista todas las operaciones
POST	/api/operaciones	Crea operación + ejecuta motor de riesgo (4 LEFT JOINS)
GET	/api/operaciones/tracking/{nro}	Busca operación por número de tracking
PUT	/api/operaciones/{id}/estado	Actualiza el estado de una operación
GET	/api/operaciones/{id}/analisis	Re-ejecuta el JOIN para operación existente
GET	/api/operaciones/alerta-roja	Lista operaciones canal ROJO no cerradas
DELETE	/api/operaciones/{id}	Elimina una operación
GET	/api/catalogos/paises-riesgo	Lista catálogo de puertos/países de riesgo
POST	/api/catalogos/paises-riesgo	Agrega un nuevo puerto al catálogo
GET	/api/catalogos/importadores	Lista importadores con historial
POST	/api/catalogos/importadores	Agrega un importador
GET	/api/catalogos/arancelarios	Lista restricciones arancelarias
POST	/api/catalogos/arancelarios	Agrega una restricción arancelaria
GET	/api/catalogos/lista-negra	Lista entidades de la lista negra global (NUEVO)
POST	/api/catalogos/lista-negra	Agrega una entidad a la lista negra (NUEVO)
POST	/api/admin/login	Autenticación (username + password)
POST	/api/admin/registro	Registra nuevo usuario admin con validaciones
GET	/api/admin/listar	Lista todos los usuarios
DELETE	/api/admin/{id}	Elimina un usuario
GET	/api/ubicaciones/paises	Lista todos los países
GET	/api/ubicaciones/provincias/{paisId}	Lista provincias de un país
GET	/api/ubicaciones/ciudades/{provId}	Lista ciudades de una provincia
GET	/api/estadisticas	Devuelve las 4 métricas del dashboard
GET	/api	Health check del servidor